Benjamin LAMY
Assim SYED MANSOOR
Mohamed GUERZOU

SCIENCES SORBONNE UNIVERSITÉ

Structures de Données

2024/2025

-

RAPPORT DE PROJET

Sommaire

I – Sujet_____2

II – Structures_____2

III – Organisation des fichiers_____3

IV – Tests et voies d'améliorations_____4

V – Initiatives et algorithmes ajoutés_____5

VI – Conclusions_____6

Benjamin LAMY
Assim SYED MANSOOR
Mohamed GUERZOU

SCIENCES SORBONNE UNIVERSITÉ

I – Sujet

Le but de ce projet est de réaliser un simulateur de CPU simplifié, capable d'exécuter un programme fourni en code pseudo-assembleur. Le simulateur doit pouvoir traiter le code brut de ce programme, gérer la mémoire nécessaire et exécuter les instructions.

II – Structures

Ce projet s'appuie sur de nombreuses structures, que voici :

- HashEntry permet de stocker une valeur générique (pointeur void*) associée à une clef dans une table de hachage générique.

```
typedef struct hashentry{
  char *key ;
  void *value ;
} HashEntry;
```

- HashMap permet de représenter une table de hachage générique. On y retrouve la taille totale de la table (dans notre cas 128) ainsi qu'un tableau de HashEntry permettant de stocker les informations dans la table.

```
typedef struct hashmap{
  int size;
  HashEntry *table ;
} HashMap ;
```

- Segment permet de représenter un segment mémoire sous forme de liste chainée. On y stocke l'adresse de début et la taille du segment sous forme d'entiers, puis un pointeur vers le segment suivant dans la mémoire.

```
typedef struct segment {
    int start;
    int size;
    struct segment* next;
} Segment;
```

- MemoryHandler permet de représenter un espace mémoire. On y trouve un tableau de pointeurs génériques, permettant de stocker les données en mémoire. On y trouve également le champ total_size (int) permettant d'indiquer la taille de la mémoire (en unités mémoire). On y trouve également une liste chainée de Segment permettant de représenter les segments libres de la mémoire. Enfin, on trouve une table de hachage permettant de représenter la mémoire allouée, chaque entrée de la table étant un Segment, et la clef le nom (unique) de ce segment.

```
typedef struct memoryHandler {
    void** memory;
    int total_size;
    Segment* free_list;
    HashMap* allocated;
} MemoryHandler;
```

- Instruction permet de stocker les informations relatives à une instruction récupérée par un parser. Les champs représentent

```
typedef struct {
  char* mnemonic;
  char* operand1;
  char* operand2;
} Instruction;
```

les différentes parties de l'instruction, à savoir le mnémonique et les deux opérandes (potentiellement vides).

- ParserResult sert à stocker les informations d'un programme parsé. Les champs data_instructions et code_instructions sont des tableaux de pointeurs sur des Instruction, représentant respectivement les instructions parsées des segments .DATA et .CODE du programme fourni. Les champs data_count et code_count permettent d'indiquer le nombre d'instructions présentent dans les tableaux respectifs. La table de hachage labels permet d'associer une étiquette (pouvant être trouvée dans une ligne du segment .CODE) au numéro de l'instruction où cette étiquette a été trouvée. Enfin, la table de hachage memory_locations permet d'associer le nom des variables du segment .DATA à leur emplacement en mémoire.

```
typedef struct {
  Instruction** data_instructions;
  int data_count;
  Instruction** code_instructions;
  int code_count;
  HashMap* labels;
  HashMap* memory_locations;
} ParserResult;
```

- CPU permet de représenter un cpu. On y trouve un MemoryHandler permettant de gérer la mémoire, une table de hachage permettant de représenter les différents registres du cpu (AX, BX, CX, DX, ZF, SF, IP, …) ainsi qu'une table de hachage constant_pool permettant un accès immédiat aux constantes numériques.

```
typedef struct {
  MemoryHandler* memory_handler;
  HashMap* context;
  HashMap* constant_pool ;
} CPU;
```

III – Organisation des fichiers

Ce projet est structuré en 4 « parties » composées chacune de 3 fichiers, permettant d'implémenter un outil spécifique nécessaire à la mise en place du cpu :

- hachageg.h et hachageg.c permettent d'implémenter une table de hachage générique.
- memory.h et memory.c permettent d'implémenter un système de mémoire. Ces fichiers incluent hachageg.h car des tables de hachage sont nécessaires pour mettre en place ce système de mémoire. Le fichier memoryTest.c, lorsqu'il est compilé, permet de tester le système d'allocation et de libération de la mémoire, à travers un menu interactif.

- parser.h et parser.c permettent d'implémenter un parser de code pseudo-assembleur. Ces fichiers incluent également le fichier hachageg.h. À l'exécution du fichier parserTest.c compilé, il est possible de visualiser le résultat du parser sur un programme préalablement chargé dans le code du fichier parserTest.c
- cpu.h et cpu.c permettent d'implémenter le cpu en lui-même, ainsi que toutes les fonctions de manipulation du cpu et d'exécution de programme. Ces fichiers incluent hachageg.h, memory.h et parser.h, ainsi que <regex.h>, nécessaire pour pouvoir utiliser les expressions régulières dans les fonctions d'adressage. Le fichier cpuTest.c permet de tester les fonctions d'initialisation du cpu et de manipulation des instructions du programme parsé, comme par exemple la substitution des variables ou labels par leur adresse, ou encore l'allocation dans la mémoire des variables et des instructions parsées.
- main.c permet de tester l'exécution d'un programme plus complexe dans le cpu (fichier pseudo_assembleur_code.inst). Ce programme permet de tester les instructions demandées dans l'énoncé, ainsi que la bonne exécution en mode pas à pas du programme. Ce fichier inclut le fichier cpu.h

Le fichier Makefile permet la compilation séparée du projet. La cible « clean » permet de nettoyer le dossier des fichiers .o et des exécutables.

IV – Tests et voies d'améliorations

Faute de temps, les algorithmes ont été implémentés selon la première forme à laquelle nous avons pensé. Nous n'avons donc pas réalisé de tests de performances temporelles afin de mesurer l'efficacité de différentes implémentations.

Les exécutions des différents exécutables du projet ont donné les résultats suivants :

- memoryTest : l'exécution de memoryTest a fourni un résultat satisfaisant. Les fonctions fonctionnent correctement, et aucune fuite mémoire n'est à déplorer.

- parserTest : l'exécution de ce fichier donne un résultat plutôt satisfaisant. Cependant, la fonction parse possède un problème : si le fichier pseudo-assembleur ne possède pas une ligne vide après la dernière instruction, cette dernière instruction n'est pas prise en compte par la fonction et n'est pas enregistrée dans le résultat du parser. L'analyse avec valgrind de cette exécution nous permet de constater qu'aucune fuite mémoire ne se produit.
- cpuTest : lors de l'exécution de ce fichier, on constate que les fonctions produisent un résultat satisfaisant. On constate également, avec valgrind, que cette exécution ne conduit à aucune fuite mémoire, cependant, on constate quelques erreurs (non fatales), qui nécessitent une correction.
- main : l'exécution de ce fichier est plus compliquée. Si on exécute le programme jusqu'au bout, le programme plante. Si on quitte le programme avant (en appuyant sur la touche q), le programme se termine correctement en apparence. Cependant, en analysant avec valgrind, on constate une fuite mémoire de plusieurs centaines d'octets. On conclut donc qu'il faut reprendre le programme pour régler, d'abord les fuites mémoires, problématiques, puis retravailler les fonctions de libération pour pouvoir finir correctement le programme, car ce sont les fonctions de libération de la mémoire qui font crasher le programme à la fin.

On peut conclure que ce projet peut encore être grandement amélioré. Entre fuites mémoires et programmes invalides, il est également possible d'optimiser certaines de nos fonctions, bien que nous ne nous soyons pas penchés dessus, faute de temps.

V – Initiatives et algorithmes ajoutés

Au cours de ce projet, il a été utile d'ajouter des algorithmes afin de nous aider, notamment :

- hachageg.h
  - hashmap_display, qui permet d'afficher une table de hachage générique, en castant les valeurs stockées en int pour les afficher.
  - isEmpty_hashmap, qui permet de décider si une table de hachage est vide ou non (cette fonction est utilisée notamment dans la fonction memory_display).

- memory.h
  - segment_init, qui permet de créer une structure Segment avec les informations renseignées lors de l'appel de la fonction
  - memory_display, qui permet d'afficher l'état de la mémoire (mémoire libre et segments alloués).
  - destroy_memory, qui permet de libérer la mémoire allouée à un espace mémoire
- parser.h
  - free_instruction, qui permet de libérer la mémoire allouée à une structure Instruction (cette fonction est utilisée ensuite pour libérer la mémoire du parser)
  - instruction_display, qui permet d'afficher les informations d'une Instruction
  - parser_display, qui permet d'afficher les données stockées dans le parser (instructions, labels et adresses des variables)
- cpu.h
  - cpu_display, qui permet d'afficher les informations relatives au cpu (registres, data_segment)

Il a également été décidé, dans les fonctions d'adressage par registre, que les registres seraient traités via des tableaux dynamiques, afin d'accepter, le cas échéant, des noms de registres dépassant les 2 caractères. Cependant, les expressions régulières n'ont pas été écrites pour accepter des formats plus longs de registres. Par ailleurs, la fonction segment_override_adressing ne supporte pas non plus un format plus long et a été implémentée pour supporter uniquement des noms de deux caractères.

VI – Conclusions

Ce projet a été le moyen de mettre en œuvre de nombreuses connaissances. Il reste cependant beaucoup de points de travail : de l'optimisation à la gestion des fuites mémoires, en passant par la généralisation à un cpu plus poussé, les vecteurs d'amélioration sont nombreux, et il sera intéressant de se pencher dessus dans le futur, pour maintenir et développer encore nos connaissances.